

Assignment 4: System Performance and Data Access Layer

Part 1: Core System Operations

Three operations were defined before writing any SQL. Each involves multiple tables and returns aggregated or non-trivial results.

Operation 1 — Admin: Revenue by Restaurant

Who	Platform admin running a revenue report.
Why	Track which restaurants earn the most and identify underperformers.
Returns	Restaurant name, total orders, total revenue — filtered by date range, sorted highest first.
Tables	ORDERS joined with RESTAURANT
Why non-trivial	Requires GROUP BY + SUM + COUNT + date range filter across completed orders.

Operation 2 — Customer: Order History

Who	A customer viewing their order history or requesting a receipt.
Why	Review past orders, verify charges, or resolve a dispute.
Returns	One row per item: order ID, date, status, restaurant, item name, quantity, price paid — newest first.
Tables	ORDERS, ORDER_ITEM, MENU_ITEM, RESTAURANT
Why non-trivial	Four-table join. Uses stored unit_price from ORDER_ITEM, not current menu price, to show what was actually charged.

Operation 3 — System: Slow Delivery Monitor

Who	An automated monitoring job or operations admin.
Why	Flag deliveries that took too long and identify drivers or restaurants causing delays.
Returns	Orders over 45 minutes: order ID, delivery time in minutes, driver name, vehicle type, restaurant — slowest first.
Tables	ORDERS, DRIVER, RESTAURANT
Why non-trivial	Delivery_minutes is computed from julianday arithmetic on two datetime columns — cannot be indexed directly.

Part 2: Naive Implementation and Diagnosis

No schema changes or indexes were added. Each query was run through EXPLAIN QUERY PLAN first.

Operation 1 — Naive

```
1  SELECT r.restaurant_id, r.name AS restaurant_name,
2  |   |   | COUNT(o.order_id) AS total_orders,
3  |   |   | ROUND(SUM(o.total_price), 2) AS total_revenue
4  FROM ORDERS o
5  JOIN RESTAURANT r ON o.restaurant_id = r.restaurant_id
6  WHERE o.status = 'completed'
7  |   AND o.order_time >= '2025-01-01' AND o.order_time <= '2025-12-31'
8  GROUP BY r.restaurant_id, r.name
9  ORDER BY total_revenue DESC;
10
```

QUERY PLAN

```
1  |--SCAN o
2  |--SEARCH r USING INTEGER PRIMARY KEY (rowid=?)
3  |--USE TEMP B-TREE FOR GROUP BY
4  ^--USE TEMP B-TREE FOR ORDER BY
```

Diagnosis: SCAN o reads every row in ORDERS because there is no index on status or order_time. SQLite cannot jump to completed orders in the date range — it reads all rows and discards non-matches. Two TEMP B-TREE structures are built for GROUP BY and ORDER BY. At 500,000 rows the SCAN becomes the dominant bottleneck.

Operation 2 — Naive

```
1  SELECT o.order_id, o.order_time, o.status, r.name AS restaurant_name,
2  |   |   | mi.name AS item_name, oi.quantity, oi.unit_price
3  FROM ORDERS o
4  JOIN ORDER_ITEM oi ON o.order_id = oi.order_id
5  JOIN MENU_ITEM mi ON oi.item_id = mi.item_id
6  JOIN RESTAURANT r ON o.restaurant_id = r.restaurant_id
7  WHERE o.customer_id = 1
8  ORDER BY o.order_time DESC;
9
```

QUERY PLAN

```
1  |--SCAN oi
2  |--SEARCH o USING INTEGER PRIMARY KEY (rowid=?)
3  |--SEARCH mi USING INTEGER PRIMARY KEY (rowid=?)
4  |--SEARCH r USING INTEGER PRIMARY KEY (rowid=?)
5  ^--USE TEMP B-TREE FOR ORDER BY
```

Diagnosis: SQLite scans all of ORDER_ITEM because there is no index on ORDERS.customer_id. It reads every order item and joins to ORDERS just to check whether it belongs to the right customer. The three primary key SEARCHes are efficient, but the initial ORDER_ITEM scan is wasteful — most rows belong to other customers.

Operation 3 — Naive

```
1  SELECT o.order_id, o.order_time, o.delivery_time,
2         ROUND((julianday(o.delivery_time) - julianday(o.order_time)) * 1440, 1) AS delivery_minutes,
3         d.name AS driver_name, d.vehicle_type, r.name AS restaurant_name
4  FROM ORDERS o
5  JOIN DRIVER d ON o.driver_id = d.driver_id
6  JOIN RESTAURANT r ON o.restaurant_id = r.restaurant_id
7  WHERE o.status = 'completed' AND o.delivery_time IS NOT NULL
8         AND (julianday(o.delivery_time) - julianday(o.order_time)) * 1440 > 45
9  ORDER BY delivery_minutes DESC;
```

QUERY PLAN

```
1  |--SCAN o
2  |--SEARCH d USING INTEGER PRIMARY KEY (rowid=?)
3  |--SEARCH r USING INTEGER PRIMARY KEY (rowid=?)
4  ^--USE TEMP B-TREE FOR ORDER BY
```

Diagnosis: SCAN o reads every row in ORDERS. With no index on status or delivery_time, the julianday expression must be evaluated for every single row before SQLite can filter. DRIVER and RESTAURANT lookups are fast primary key searches, but the full table scan on ORDERS is the expensive step.

Part 3: Optimization Through Design

Each operation was improved using a design change (a new view), not just a query rewrite.

Operation 1 — Pre-aggregated Revenue View

What changed: Created `vw_restaurant_revenue_summary`, which pre-aggregates completed orders by restaurant and month. The outer query then sums the monthly buckets — a much smaller dataset than raw order rows.

```
1 CREATE VIEW vw_restaurant_revenue_summary AS
2 SELECT o.restaurant_id, r.name AS restaurant_name,
3        strftime('%Y-%m', o.order_time) AS order_month,
4        COUNT(o.order_id) AS monthly_orders,
5        ROUND(SUM(o.total_price), 2) AS monthly_revenue
6 FROM ORDERS o JOIN RESTAURANT r ON o.restaurant_id = r.restaurant_id
7 WHERE o.status = 'completed' AND o.total_price IS NOT NULL
8 GROUP BY o.restaurant_id, r.name, strftime('%Y-%m', o.order_time);
```

-- Optimized query

```
1 SELECT restaurant_name, SUM(monthly_orders) AS total_orders,
2        SUM(monthly_revenue) AS total_revenue
3 FROM vw_restaurant_revenue_summary
4 WHERE order_month >= '2025-01' AND order_month <= '2025-12'
5 GROUP BY restaurant_name ORDER BY total_revenue DESC;
```

QUERY PLAN

```
1 |--CO-ROUTINE vw_restaurant_revenue_summary
2 | |--SCAN o
3 | |--SEARCH r USING INTEGER PRIMARY KEY (rowid=?)
4 | `--USE TEMP B-TREE FOR GROUP BY
5 |--SCAN vw_restaurant_revenue_summary
6 |--USE TEMP B-TREE FOR GROUP BY
7 `--USE TEMP B-TREE FOR ORDER BY
```

What improved: The outer query works on pre-grouped monthly rows instead of raw order rows. The view is also reusable — an admin can query it for a month-by-month breakdown without re-aggregating from scratch. When this view is later materialized, the outer query becomes a trivial lookup.

Operation 2 — Reuse vw_customer_order_history (Assignment 3)

What changed: The naive query was replaced with a query against the Assignment 3 view, vw_customer_order_history. Combined with the customer_id index added in Part 4, SQLite now starts from ORDERS rather than scanning ORDER_ITEM.

```
1  SELECT order_id, order_time, status, restaurant_name, item_name, quantity, unit_price
2  FROM vw_customer_order_history
3  WHERE customer_id = 1
4  ORDER BY order_time DESC;
```

QUERY PLAN (after idx_orders_customer_id added in Part 4)

```
1  |--SEARCH o USING INDEX idx_orders_customer_id (customer_id=?)
2  |--SEARCH r USING INTEGER PRIMARY KEY (rowid=?)
3  |--SEARCH oi USING INDEX sqlite_autoindex_ORDER_ITEM_1 (order_id=?)
4  |--SEARCH mi USING INTEGER PRIMARY KEY (rowid=?)
5  ^--USE TEMP B-TREE FOR ORDER BY
```

What improved: The ORDER_ITEM SCAN is eliminated. All four table accesses are now SEARCH operations. SQLite finds the customer's orders first, then looks up only their items.

Operation 3 — vw_delivery_performance View

What changed: Created vw_delivery_performance, which encapsulates the joins and pre-computes delivery_minutes. The outer query filters on that column. The time calculation now lives in one place and only needs to be changed in the view.

```
1  CREATE VIEW vw_delivery_performance AS
2  SELECT o.order_id, o.order_time, o.delivery_time,
3         ROUND((julianday(o.delivery_time) - julianday(o.order_time)) * 1440, 1) AS delivery_minutes,
4         d.name AS driver_name, d.vehicle_type, r.name AS restaurant_name
5  FROM ORDERS o
6  JOIN DRIVER d ON o.driver_id = d.driver_id
7  JOIN RESTAURANT r ON o.restaurant_id = r.restaurant_id
8  WHERE o.status = 'completed' AND o.delivery_time IS NOT NULL;
```

-- Optimized query

```
1  SELECT * FROM vw_delivery_performance
2  WHERE delivery_minutes > 45 ORDER BY delivery_minutes DESC;
3
4  QUERY PLAN (after idx_orders_status_time added in Part 4)
5  |--SEARCH o USING INDEX idx_orders_status_time (status=?)
6  |--SEARCH d USING INTEGER PRIMARY KEY (rowid=?)
7  |--SEARCH r USING INTEGER PRIMARY KEY (rowid=?)
8  ^--USE TEMP B-TREE FOR ORDER BY
```

What improved: The SCAN on ORDERS is now a SEARCH — cancelled and pending orders are never read. The view keeps the delivery_minutes logic centralized for future maintenance.

Part 4: Indexing

Three indexes total. Each justified with before-and-after EXPLAIN output.

Index 1 — `idx_orders_status_time` ON `ORDERS(status, order_time)`

Column choice: Operations 1 and 3 both filter `WHERE status='completed'` with a date range. The composite index with `status` first handles the equality filter, and `order_time` handles the range scan.

Before:

QUERY PLAN

```
`--SCAN ORDERS
```

After:

QUERY PLAN

```
`--SEARCH ORDERS USING INDEX idx_orders_status_time (status=? AND order_time>?)
```

Why it matters: `SCAN` reads every row. `SEARCH` jumps directly to matching rows. This index alone eliminates full table scans for two of the three operations.

Index 2 — `idx_orders_customer_id` ON `ORDERS(customer_id)`

Column choice: Operation 2 filters `WHERE customer_id = ?`. Without this, SQLite scans all of `ORDER_ITEM` and joins to `ORDERS` to find the right rows.

Before:

QUERY PLAN

```
`--SCAN ORDERS
```

After:

QUERY PLAN

```
`--SEARCH ORDERS USING COVERING INDEX idx_orders_customer_id (customer_id=?)
```

Why it matters: SQLite reports this as a `COVERING INDEX` — the index alone answers the lookup without touching the main table. The access pattern flips: `ORDERS` is searched first, then `ORDER_ITEM` items are retrieved by primary key.

Index 3 — `idx_orders_restaurant_id` ON `ORDERS(restaurant_id)`

Column choice: When filtering or grouping by a specific restaurant, this index eliminates a full scan. It also supports the `GROUP BY restaurant_id` in Operation 1 and the restaurant owner access pattern from Assignment 3.

Before:

QUERY PLAN

```
`--SEARCH o USING INDEX idx_orders_status_time (status=?)
```

After:

QUERY PLAN

```
`--SEARCH o USING INDEX idx_orders_restaurant_id (restaurant_id=?)
```

Why it matters: Restaurant-scoped queries were previously unindexed. ORDER_ITEM already has a built-in autoindex on (order_id, item_seq), so spending this budget slot there would be redundant. The restaurant_id index fills a real gap.

Part 5: Python Access Layer

Three functions in assignment4.py. Each accepts parameters, runs the optimized query, and returns a list of dictionaries. All functions use sqlite3.Row so columns are accessible by name.

Function 1 — get_revenue_by_restaurant(start_date, end_date)

```
1 def get_revenue_by_restaurant(start_date="2025-01-01", end_date="2025-12-31"):
2     query = """
3         SELECT r.restaurant_id, r.name AS restaurant_name,
4             COUNT(o.order_id) AS total_orders,
5             ROUND(SUM(o.total_price), 2) AS total_revenue
6         FROM ORDERS o JOIN RESTAURANT r ON o.restaurant_id = r.restaurant_id
7         WHERE o.status = 'completed'
8             AND o.order_time >= ? AND o.order_time <= ?
9         GROUP BY r.restaurant_id, r.name ORDER BY total_revenue DESC;"""
10    with get_connection() as conn:
11        rows = conn.execute(query, (start_date, end_date + " 23:59:59")).fetchall()
12    return [dict(row) for row in rows]
13
```

Sample output:

```
[{'restaurant_name': 'Sushi World', 'total_orders': 4, 'total_revenue': 102.93},
{'restaurant_name': 'Pizza Heaven', 'total_orders': 4, 'total_revenue': 94.93},
{'restaurant_name': 'Burger Palace', 'total_orders': 4, 'total_revenue': 87.90}, ...]
```

Function 2 — get_customer_order_history(customer_id)

```
1 def get_customer_order_history(customer_id):
2     query = """
3         SELECT order_id, order_time, status, restaurant_name,
4             item_name, quantity, unit_price
5         FROM vw_customer_order_history
6         WHERE customer_id = ? ORDER BY order_time DESC;"""
7     with get_connection() as conn:
8         rows = conn.execute(query, (customer_id,)).fetchall()
9     return [dict(row) for row in rows]
10
```

Sample output (customer_id=1):

```
[{'order_id': 16, 'order_time': '2025-02-06 11:00', 'status': 'pending',
'restaurant_name': 'Sushi World', 'item_name': 'California Roll',
'quantity': 1, 'unit_price': 12.99}, ...]
```

Function 3 — get_slow_deliveries(threshold_minutes, start_date, end_date)

```
1 def get_slow_deliveries(threshold_minutes=45, start_date=None, end_date=None):
2     query = """SELECT order_id, order_time, delivery_time, delivery_minutes,
3         driver_name, vehicle_type, restaurant_name
4         FROM vw_delivery_performance WHERE delivery_minutes > ?"""
5     params = [threshold_minutes]
6     if start_date: query += " AND order_time >="; params.append(start_date)
7     if end_date:   query += " AND order_time <="; params.append(end_date + " 23:59:59")
8     query += " ORDER BY delivery_minutes DESC;"
9     with get_connection() as conn:
10        rows = conn.execute(query, params).fetchall()
11    return [dict(row) for row in rows]
12
```

Sample output:

```
[{'order_id': 7, 'delivery_minutes': 70.0, 'driver_name': 'Marcus Lee',
  'vehicle_type': 'Car', 'restaurant_name': 'Sushi World'},
 {'order_id': 13, 'delivery_minutes': 65.0, 'driver_name': 'Marcus Lee', ...}]
```

Part 6: Reflection

Which operation was hardest to optimize?

Operation 3 was the hardest to optimize. The `delivery_minutes` value is calculated using julianday arithmetic on two datetime columns, which means SQLite has no way to index it directly. The `idx_orders_status_time` index helped a lot by cutting down how many rows even reach that calculation, but the math still has to run on every completed order that passes the filter. The real fix would be adding a `delivery_minutes` column to the table and using a trigger to populate it whenever `delivery_time` gets set. Then it could be indexed just like any other column.

What made the biggest difference?

Indexing made the biggest difference overall. Just adding `idx_orders_status_time` turned a full table scan into a targeted search for two out of three operations, which is a completely different situation performance-wise. Views were helpful too since they cleaned up the join logic and made queries easier to reuse, but a view by itself does not help if the underlying tables are still being scanned. Using both together was the right call: the views kept things organized and the indexes actually made them fast.

What happens to Operation 1 at 1 million orders?

Operation 1 would be the first to break at a million orders. The index helps narrow down the date range, but SQLite still has to group and sort however many completed orders fall in that window, and both `GROUP BY` and `ORDER BY` require temporary structures built in memory. The view pre-groups by month which helps, but it still reruns the full aggregation every time it is called. At that scale the real solution would be a summary table that gets updated on a nightly schedule, so the query is just reading a few pre-calculated rows instead of crunching through hundreds of thousands of them.

Did this assignment change how you think about the Assignment 2 schema?

Yes, the `REVIEW` table stood out right away because it has no `restaurant_id` or `order_id`, so there is no straightforward way to calculate per-restaurant ratings or tie a review to a specific order. I also noticed that `ORDERS.status` is just stored as plain text with no constraint on what values are allowed. The trigger from Assignment 3 handles transitions at the application level, but a `CHECK` constraint would block bad values at the database level no matter what. The last thing was the missing `delivery_minutes` column. Having to compute it on the fly in every query was avoidable and made Operation 3 permanently harder to work with than it needed to be.